

Research Paper

Engineering Trustworthy AI Assignment Marking Suggestions for a Deterministic Student Portal

A design science case study using AWS services contracts human oversight and measurable evidence

Item	Description
Case study	IdeaFarm Secure Code Lecturer and Student Portal AI-assisted marking suggestions
Research focus	Controlling probabilistic model behavior inside a deterministic academic workflow
Primary decision	AI recommends; deterministic services and lecturers authorize official outcomes
Architecture context	Amazon Bedrock and event-driven AWS application services
Date	17 September 2026

This paper presents a governed architecture for using generative AI to prepare evidence-grounded marking suggestions without allowing a probabilistic model to become the academic system of record. The design is tuned against the IdeaFarm Secure Code Lecturer repository, a working AWS serverless prototype for ZIP submission, asynchronous Bedrock Agent grading, CodeQL-supported analysis, result storage, and instructor views. The central conclusion is that prompt engineering alone cannot make marking deterministic. Trust arises from a layered contract: immutable submissions and rubrics, schema-constrained suggestions, deterministic arithmetic and authorization, independent evaluation, safe failure, and lecturer approval before publication.

Research status. The work is a design science case study and implementation proposal. It does not claim results from a completed institutional trial. The evaluation thresholds and experiments are explicit hypotheses for validation before operational use.

Abstract

Large language models can reduce the effort required to locate evidence in student work and draft formative feedback, but they remain probabilistic. Repeated runs may produce different scores or rationales; models can hallucinate evidence, misapply a rubric, exhibit model-specific severity, and respond unpredictably to irrelevant or adversarial content. These properties conflict with deterministic requirements for academic assessment, including stable policy, attributable decisions, privacy, reproducibility, and appeal.

This paper proposes a Student Portal architecture in which generative AI produces a marking suggestion rather than an official mark. The design uses authenticated and immutable submission intake, a versioned rubric and assessment manifest, asynchronous orchestration, structured Bedrock output, citation and arithmetic validation, bounded confidence rules, lecturer approval, and a complete audit trail. It defines business invariants, OpenAPI and event contracts, failure states, evaluation datasets, release gates, production metrics, and failure-injection tests. The design aligns with the NIST AI Risk Management Framework Generative AI Profile and the AWS Well-Architected Generative AI Lens. Research on automated grading supports the potential of LLM assistance while also showing moderate human alignment, cross-model differences, drift, and recognition failures, which justifies a human-accountable recommendation model rather than autonomous marking [1-4].

The contribution is a practical pattern for placing a probabilistic component inside a deterministic business application: the model proposes a typed evidence pack; deterministic controls decide whether the pack is valid; an authorized lecturer decides whether the academic outcome may be published.

Keywords: generative AI; automated assessment; assignment marking; deterministic systems; Amazon Bedrock; human oversight; design first; evidence contract; education technology

Contents

- 1 Introduction
- 2 Research problem and questions
- 3 Evidence from research and standards
- 4 Student Portal case study
- 5 Why the AI component is probabilistic
- 6 Design principles and business invariants
- 7 Proposed AWS architecture
- 8 Contracts and deterministic controls
- 9 Detailed marking workflow
- 10 Evaluation and release methodology

11 Failure injection and operational evidence

12 Security privacy and fairness

13 Implementation roadmap

14 Discussion limitations and conclusion

References

Appendix A Contract examples

Appendix B Acceptance criteria

1 Introduction

Assignment marking combines interpretation and authority. Interpretation involves reading a submission, mapping evidence to rubric criteria, identifying misconceptions, and drafting useful feedback. Authority involves deciding an official score, applying institutional policy, releasing the result, and handling exceptions or appeals. Generative AI may assist with the first group of tasks, but an unconstrained model is not a suitable authority for the second.

The distinction matters because a fluent response can conceal an unsupported inference. In an ordinary chatbot, a wrong sentence may cause inconvenience. In an assessment workflow, a wrong mark affects progression, confidence, workload, fairness, and institutional accountability. The Student Portal must therefore treat every model output as untrusted candidate data until deterministic and human controls accept it.

The paper develops a concrete architecture for a portal in which students submit assignments, lecturers configure rubrics, and Amazon Bedrock prepares structured marking suggestions. The architecture is “deterministic” not because every model output is identical, but because the surrounding application produces predictable state transitions and never permits uncertain output to bypass authority. The same accepted submission and rubric always retain the same immutable identity; required validation gates always run; arithmetic is reproducible; roles and permissions are explicit; invalid output always enters a documented failure state; and only a lecturer can publish the official result.

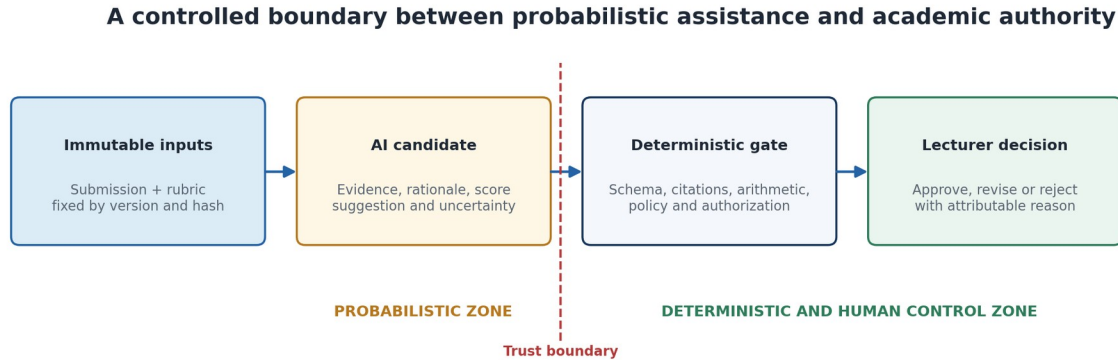


Figure 1. The model operates inside a bounded suggestion zone. Deterministic validation and an accountable lecturer prevent uncertain output from becoming an official academic decision.

2 Research Problem and Questions

The design problem is: how can a Student Portal obtain the productivity and pedagogical benefits of generative AI while preserving the deterministic properties required by an academic assessment system?

Research question	Operational interpretation
RQ1	Which parts of assignment marking are suitable for probabilistic assistance, and which must remain

	deterministic or human-controlled?
RQ2	Which contracts and evidence gates prevent hallucinated, inconsistent, unauthorized, or mathematically invalid suggestions from changing official records?
RQ3	How should quality, consistency, fairness, security, latency, cost, and recovery be measured before and after release?
RQ4	How should the application behave when the model, retrieval layer, or downstream services are slow, unavailable, or uncertain?
RQ5	What audit evidence enables lecturers to understand, approve, override, reproduce, and defend the final decision?

The scope is marking suggestions for submitted assignments. The system may extract evidence, propose criterion scores, identify missing requirements, and draft feedback. It does not autonomously publish official marks, make misconduct findings, change deadlines, grant exemptions, or resolve appeals.

3 Evidence from Research and Standards

3.1 Research on AI assisted grading

Recent work gives a mixed but actionable picture. CodEv reports that rubric alignment, model ensembles, and agreement tests can produce useful and consistent programming feedback under controlled conditions [1]. A large comparison of 18 models over more than 6,000 programming submissions found systematic differences in strictness and only moderate agreement with human teachers, showing that model choice is not neutral [2]. A case study of advanced software engineering assessment reported intra-model and inter-model inconsistency and warned that interaction history can shift grading standards [3]. A multimodal study of authentic handwritten STEM submissions found latent recognition failures that can propagate into grading, while targeted human intervention improved robustness [4].

The practical implication is not that AI has no place in marking. It is that reliability must be engineered and continuously demonstrated for the exact task, rubric, submission type, model version, and operating context. The application should reduce lecturer search and drafting effort without converting uncertain inference into unreviewed authority.

3.2 NIST and AWS guidance

NIST AI 600-1 extends the AI Risk Management Framework for generative AI and emphasizes governance, measurement, incident communication, content provenance, and risk treatment across the lifecycle [5]. The AWS Well-Architected Generative AI Lens similarly frames quality evaluation, tracing, security, resilience, cost, and responsible operation as architecture concerns rather than prompt-only concerns [6].

Amazon Bedrock Guardrails provides content, denied-topic, sensitive-information, contextual-grounding, and Automated Reasoning controls. AWS documentation explicitly notes that Automated Reasoning checks validate outputs against defined policies and return findings that the application must act upon; they are not a complete prompt-injection defense and apply only within the policy scope [7,8]. This supports a layered design in which automated reasoning may verify formal academic rules, while ordinary code still owns authorization, arithmetic, workflow state, and publication.

Evidence	Design implication
Cross-model and human disagreement	Calibrate and approve each model and rubric combination; do not assume interchangeability.
Repeated-run variability	Measure score variance and criterion agreement; route unstable cases to review.
Recognition and extraction errors	Separate extraction confidence from grading; provide lecturers the original artifact and cited passages.
Hallucinated rationale	Require exact evidence citations and reject unsupported claims.

Policy scope limitations	Keep deadlines, enrollment, arithmetic, role checks, and publication logic in deterministic services.
Human oversight findings	Make lecturer approval a mandatory state transition, not an optional quality check.

4 Student Portal Case Study

4.1 Business context

The Student Portal supports assignment publication, student submission, marking, feedback, and result release. Peak traffic occurs near deadlines. Students require immediate, trustworthy receipts. Lecturers need assistance locating rubric evidence and drafting feedback, but remain accountable for official results. Administrators require auditability, privacy, role separation, and the ability to reproduce what information informed a decision.

Actor	Permitted actions	Prohibited AI substitution
Student	Submit work, view own status and released feedback, raise a query.	The model cannot establish identity, change the submission, or decide an appeal.
Lecturer	Approve rubric, review evidence, adjust suggestions, publish marks, record override reasons.	Approval cannot be inferred from silence or model confidence.
Module coordinator	Version rubrics, approve assessment configuration, review quality and fairness evidence.	The model cannot redefine criteria or weights.
Portal services	Authorize, validate, persist, calculate totals, enforce state transitions, audit.	Deterministic controls cannot be delegated to prompt text.
Bedrock model	Produce a structured evidence-linked suggestion and draft feedback.	It cannot publish marks, alter source artifacts, or call unrestricted tools.

4.2 Priority use case

1. The student authenticates and submits an assignment with an idempotency key.
2. The portal validates module enrollment, deadline policy, file type, size, malware status, and ownership.
3. The portal stores an immutable S3 version, calculates a content hash, writes an ACCEPTED record, and returns a receipt.
4. An event-driven workflow extracts text and code, retrieves the approved assignment brief and rubric, and invokes the selected Bedrock model.
5. The model returns a schema-constrained criterion evidence pack with suggested scores, citations, uncertainty, and feedback.
6. Deterministic services validate schema, citation existence, score ranges, weight arithmetic, policy constraints, and authorization.
7. A lecturer compares the suggestion with the original submission and rubric, then approves, edits, or rejects it.
8. Only the lecturer approval transition writes the official result and releases feedback to the student.

4.3 Business invariants

- Every accepted submission maps to exactly one immutable and attributable artifact version.
- The active assignment brief and rubric version are fixed for each assessment job.
- Every criterion suggestion cites evidence from the submitted artifact or approved rubric.
- The sum and weighting of criterion scores are calculated by deterministic code.
- A model output never becomes an official mark without lecturer approval.
- Every approval, edit, rejection, override, and release is attributable and auditable.
- A student can access only their own submission, status, and released outcome.

4.4 Real project baseline IdeaFarm Secure Code Lecturer

The case study is grounded in the public IdeaFarm Secure Code Lecturer repository at commit 1e387b54a0eba7cb18bc1a887f1d35c78849f7d0, dated 20 July 2025 [11]. The repository demonstrates an executable serverless foundation rather than a purely conceptual architecture. Its Streamlit interface supports ZIP upload, cleaning and preview, token estimation, status polling, report dashboards, editable grading rules, GitHub CodeQL workflows, and Cognito login navigation. The AWS implementation contains presigned S3 upload logic, SQS-triggered Lambda grading, Amazon Bedrock Agent invocation, DynamoDB metadata, S3 reports, API Gateway endpoints, CloudFormation, test notes, performance observations, and a separate AWS WAF CDK stack.

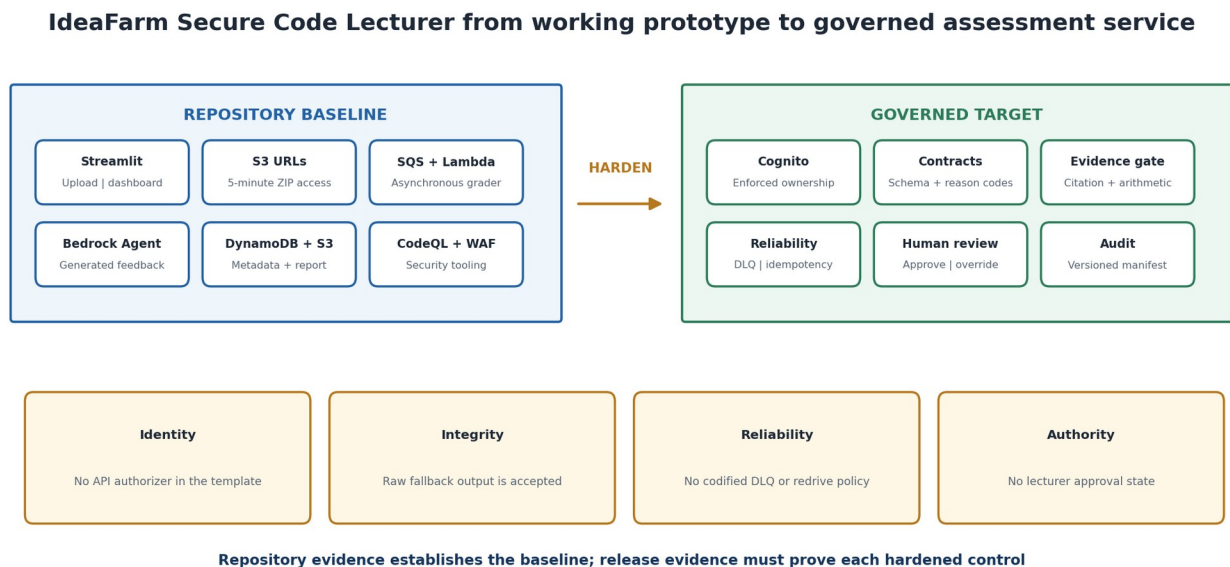


Figure 2. Evidence-based evolution of the real IdeaFarm prototype. The proposed controls preserve the existing event-driven design while adding enforceable identity, deterministic validation, reliability, human authority, and audit evidence.

Repository evidence	Observed implementation	Research-paper tuning
Submission interface	A Lambda generates five-minute S3 presigned URLs for ZIP objects and records uploader metadata when	Require Cognito authorization, owner-bound object keys, content-length and checksum conditions,

	identity claims are available.	malware scanning, idempotency, and an immutable receipt.
Asynchronous grading	Documentation describes S3 to EventBridge to SQS to Lambda; the grader downloads Python and JavaScript files and invokes a Bedrock Agent.	Codify the complete event route, queue policy, DLQ, redrive, visibility timeout, concurrency, replay protection, and terminal-state reconciliation in infrastructure as code.
AI output	The grader attempts JSON parsing but retains raw unstructured feedback when parsing fails.	Fail closed on schema violations; require criterion identifiers, evidence citations, uncertainty, manifest versions, and deterministic reason codes.
Persistence and retrieval	Reports are stored in S3 and submission metadata in DynamoDB; polling returns a short-lived report URL.	Enforce owner and lecturer authorization, conditional state transitions, versioning, KMS keys, retention, and an approval state before student release.
Security controls	The repository includes CodeQL workflows, Cognito UI navigation, presigned URLs, WAF CDK, rate limiting, and CloudWatch-oriented test guidance.	Attach an authorizer to every API method, replace the shared broad Lambda role with per-function roles, restrict Bedrock resources, redact logs, and integrate WAF with the deployed API stage.
Verification	Markdown tests cover Lambda behavior, SQS triggering, visibility-timeout reasoning, CloudWatch inspection, DynamoDB records, and S3 reports.	Convert manual checks into automated contract, security, load, replay, failure-injection, and model-quality tests with retained evidence.

5 Why the AI Component Is Probabilistic

A generative model estimates likely output tokens from its learned parameters and supplied context. Small changes in prompt construction, context order, decoding, model revision, system configuration, or conversation history may change the response. Even with conservative decoding, identical-looking requests do not create a mathematical guarantee of identical academic judgment.

Risk	Student Portal manifestation	Required control
Inaccuracy	The model overlooks a correct implementation or misreads a requirement.	Golden-set evaluation, citations, lecturer review, and protected source display.
Inconsistency	Repeated runs assign different criterion scores or reasons.	Variance tests, fixed manifests, bounded review thresholds, and no autonomous publication.
Hallucination	Feedback refers to code, evidence, or errors not present in the submission.	Citation resolution and claim validation; unsupported claims fail the gate.
Model-specific severity	One model consistently marks more harshly or leniently.	Human calibration, model comparison, subgroup analysis, and version approval.
Context contamination	Prior students, long histories, or retrieved text alter judgment.	Stateless assessment jobs, tenant isolation, scoped retrieval, and clean prompts.
Prompt injection	Submission text attempts to change the rubric or reveal instructions.	Treat submission as data, isolate system instructions, apply filters, and prohibit tool authority.
Extraction error	OCR or parsing corrupts handwritten, diagram, or code evidence.	Expose source fidelity, confidence, and original artifact; route low confidence to review.
Latency and availability	Inference is slow or unavailable near the deadline.	Durable asynchronous workflow, bounded retries, DLQ, pending status, and manual marking fallback.

The design goal is therefore controlled uncertainty. The system need not force a language model to behave like a database. It must ensure that variation cannot cross a deterministic boundary without validation and accountable approval.

6 Design Principles and Business Invariants

Principle	Application
AI recommends; people authorize	Bedrock produces a suggestion. A lecturer publishes the

	official result.
Facts before fluency	The workflow binds the exact submission, rubric, brief, and versions before inference.
Structured output before rendering	The model returns typed criterion objects, not an unbounded essay.
Evidence before score	Every proposed score includes cited evidence and criterion rationale.
Code owns arithmetic and state	Weights, totals, caps, status transitions, idempotency, and permissions remain deterministic.
Failure is visible	Invalid or uncertain output becomes RETRY, REVIEW, or FAILED, never silent completion.
Acceptance requires evidence	A release is approved only when documented quality, fairness, security, latency, cost, and recovery thresholds pass.
Version everything relevant	Submission, rubric, prompt, model, guardrail, retrieval corpus, parser, schema, and rules form one manifest.

7 Proposed AWS Architecture

The architecture separates deterministic intake, probabilistic analysis, deterministic validation, and human authority. The synchronous submission path ends after durable acceptance. Model latency never determines whether a student receives a valid receipt.

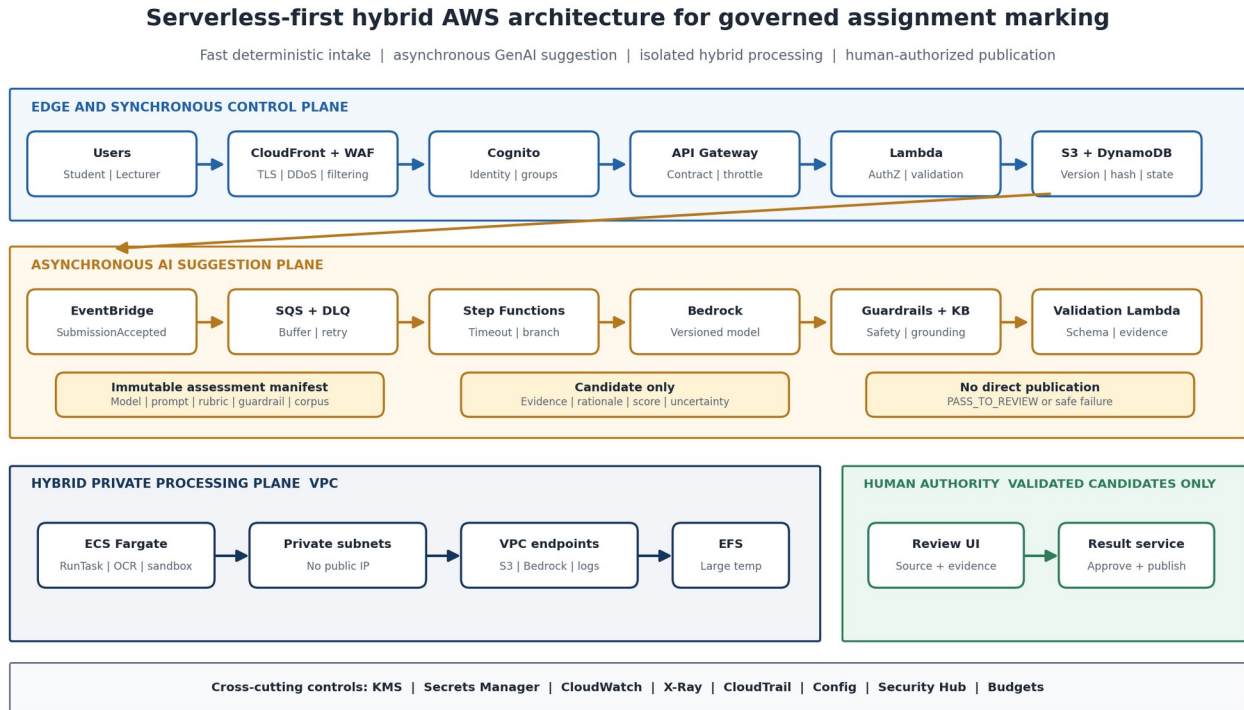


Figure 3. Serverless-first hybrid architecture. Managed services handle the bursty default path; private Fargate processing is reserved for long-running or dependency-heavy work and cannot publish an official result.

Layer	AWS services	Responsibility
Experience	Amazon CloudFront, AWS WAF, Student Portal SPA	Secure delivery, upload initiation, visible status, review interface.
Identity and API	Amazon Cognito, Amazon API Gateway, AWS Lambda	Authentication, role checks, enrollment and deadline policy, schema and idempotency validation.
System of record	Amazon S3 Versioning, Amazon DynamoDB, AWS KMS	Immutable artifact versions, hashes, job state, manifests, encryption, audit references.
Events and orchestration	Amazon EventBridge, Amazon SQS, AWS Step Functions	Durable events, burst buffering, retries, deadlines, branching, DLQ and terminal states.
AI and grounding	Amazon Bedrock, optional Knowledge Bases, Bedrock Guardrails	Evidence extraction and feedback candidate under a versioned prompt, model, and safety configuration.
Deterministic gate	AWS Lambda or containerized	JSON Schema, citation resolution,

	validation service	score bounds, arithmetic, policy checks, and release eligibility.
Observability and security	Amazon CloudWatch, AWS X-Ray, AWS CloudTrail, Security Hub	Correlated traces, alarms, audit, incident investigation, and control evidence.

7.1 Why serverless first and where hybrid processing belongs

The design uses managed, event-driven services for the default path because submission traffic is bursty, durable acceptance must be fast, and idle capacity should be minimal. Lambda owns short deterministic operations such as authorization, schema validation, idempotency, state changes, and evidence checks. A private ECS Fargate processing plane is invoked only for workloads that exceed practical Lambda boundaries, including long-running OCR, large archive expansion, controlled code execution, specialist native libraries, or sustained batch processing. Fargate tasks receive a scoped job identity, run without a public IP, use VPC endpoints, write results to versioned storage, and never gain permission to publish an official mark.

Workload	Preferred runtime	Architecture reason
Submission receipt and status	API Gateway and Lambda	Low-latency, burst-tolerant, deterministic and scale-to-zero request path.
Events, retry and orchestration	EventBridge, SQS and Step Functions	Durable decoupling, bounded retry, timeout, branching and visible terminal states.
Model inference and grounding	Amazon Bedrock, Guardrails and optional Knowledge Bases	Managed model access with versioned configuration, safety controls and grounded context.
Schema, citation and arithmetic validation	Lambda	Short deterministic checks with explicit reason codes and no model authority.
Large OCR, archive analysis or code sandbox	ECS Fargate in private subnets	Long duration, custom dependencies, resource isolation and controlled concurrency.
Official decision and publication	Lecturer review UI and deterministic result service	Separates suggestion from academic authority and preserves attributable approval.

7.2 Well Architected Framework and Generative AI Lens alignment

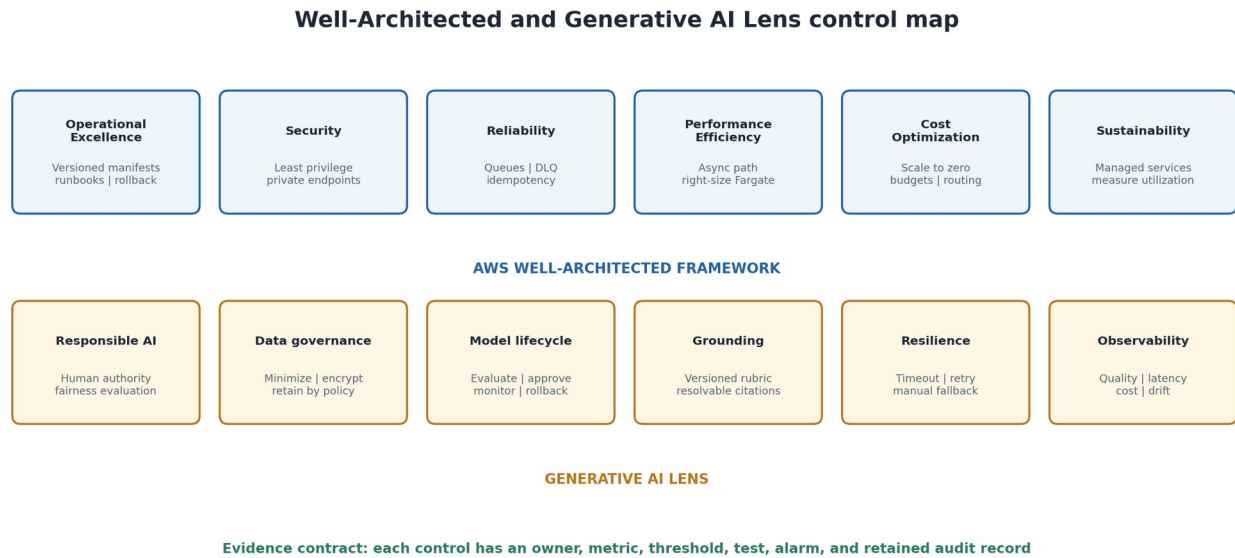


Figure 4. The six Well-Architected pillars govern the workload as a whole, while the Generative AI Lens adds model, data, grounding, responsible-AI, resilience, and quality controls for the probabilistic component.

Perspective	Key design decisions	Measurable evidence
Operational Excellence	Versioned infrastructure, prompt, model, rubric and guardrail manifests; runbooks; canary release and rollback.	Deployment record, manifest diff, runbook test, rollback recovery time and change failure rate.
Security	Cognito and backend authorization, KMS, private subnets and endpoints, least-privilege job roles, prompt isolation.	Denied cross-student tests, IAM analysis, CloudTrail trace, encryption checks and prompt-attack results.
Reliability	Immutable intake, idempotency, SQS buffering, DLQ, Step Functions timeouts, reconciliation and manual fallback.	No lost accepted jobs, duplicate suppression, oldest-message age, terminal-state reconciliation and recovery time.
Performance Efficiency	Synchronous receipt separated from inference; model routing; concurrency controls; Fargate only for justified workloads.	P95 receipt latency, end-to-end suggestion latency, queue wait, token use and task utilization.
Cost Optimization	Scale-to-zero services, per-job cost attribution, budget alarms, model selection and bounded retries.	Cost per accepted and lecturer-approved suggestion, retry cost and monthly budget variance.
Sustainability	Managed services, workload right-sizing, artifact lifecycle and avoidance of wasteful repeated inference.	Compute utilization, token consumption, cache or reuse rate and retained-storage trend.
Generative AI Lens	Grounded citations, evaluation	Citation precision, hallucination rate,

	dataset, uncertainty routing, human oversight, drift monitoring and controlled model lifecycle.	repeated-run stability, override rate, subgroup analysis and rollback evidence.
--	---	---

7.3 Logical flow

Student -> Cognito -> API Gateway and Lambda -> S3 Versioning and DynamoDB -> EventBridge -> SQS -> Step Functions -> extraction and rubric retrieval -> Bedrock and Guardrails -> deterministic evidence gate -> lecturer review -> official result service -> student notification.

7.4 State machine

State	Entry condition	Permitted next states
ACCEPTED	Submission stored, hashed, authorized, and attributable.	QUEUED, CANCELLED_BY_POLICY
QUEUED	Durable event and job identity recorded.	PROCESSING, FAILED_RETRYABLE
PROCESSING	Extraction and AI workflow running.	VALIDATING, FAILED_RETRYABLE, FAILED_FINAL
VALIDATING	Candidate returned and deterministic gates running.	LECTURER_REVIEW, RETRY, REJECTED
LECTURER_REVIEW	Evidence pack valid enough for accountable review.	APPROVED, OVERRIDDEN, REJECTED
APPROVED or OVERRIDDEN	Lecturer identity and reason recorded.	PUBLISHED
PUBLISHED	Official outcome written and released.	APPEAL_OPEN; no silent mutation

Assessment lifecycle with safe failure states

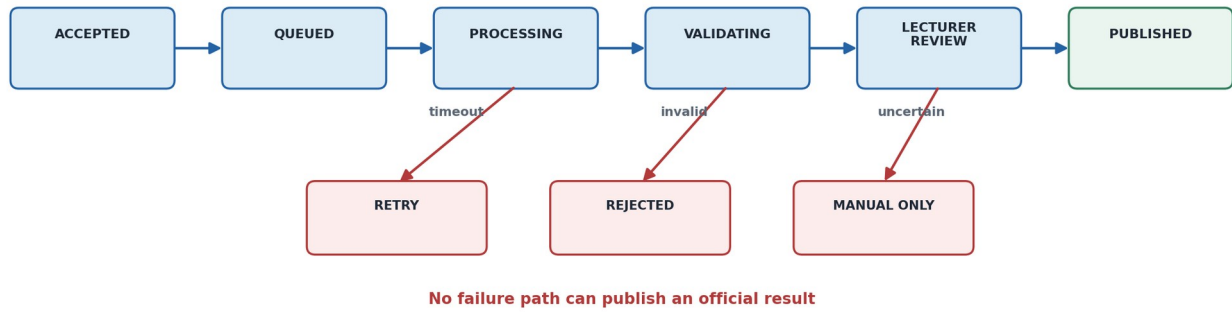


Figure 5. Lifecycle and safe-failure model. Retry, rejection, and manual-only routes remain visible and cannot reach publication without lecturer approval.

8 Contracts and Deterministic Controls

8.1 Submission interface contract

POST /assignments/{assignmentId}/submissions requires an authenticated student, Idempotency-Key, artifact metadata, and content hash. A 202 response means the artifact and submission record are durably accepted; it does not mean AI feedback is complete. Reuse of the same key with the same payload returns the known submission. Reuse with a conflicting payload returns 409.

8.2 AI suggestion contract

Field	Rule
assessmentJobId	Immutable job identifier linked to one submission and manifest.
criterionId	Must resolve to the active rubric version.
suggestedScore	Numeric value inside the deterministic criterion range.
evidence[]	One or more citations with artifact location, excerpt hash, and source type.
rationale	Explanation limited to cited evidence and rubric language.
uncertainty	Enumerated LOW, MEDIUM, HIGH with machine-readable reason codes.
feedbackDraft	Formative language; cannot contain an official publication directive.
manifest	Model ID, inference profile, prompt, rubric, parser, schema, guardrail, and corpus versions.

8.3 Deterministic evidence gate

9. Parse against JSON Schema; reject extra, missing, or malformed fields.
10. Resolve every criterion ID and citation against the immutable rubric and submission.
11. Verify excerpt hashes and ensure cited evidence belongs to the current student and artifact version.
12. Recalculate criterion and total scores using deterministic decimal arithmetic.
13. Apply caps, mandatory components, late policy, and module rules from versioned policy code.
14. Run content safety, sensitive-data, and prompt-attack checks.
15. Compare uncertainty, citation coverage, repeated-run agreement, and configured review thresholds.
16. Emit PASS_TO_REVIEW, RETRY, REJECT, or MANUAL_ONLY with explicit reason codes.

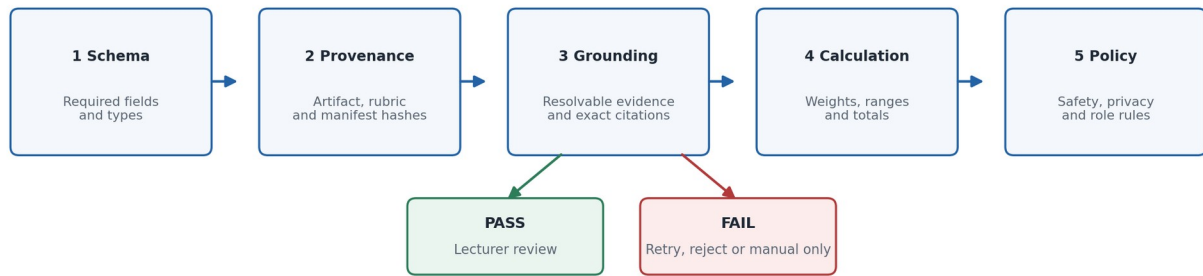
Deterministic evidence gate for every AI suggestion

Figure 6. The deterministic evidence gate converts a fluent model response into a typed, traceable candidate. Failure at any gate produces an explicit safe state.

Automated Reasoning checks may validate clear academic policies, such as eligibility conditions or rubric combinations, but application code must inspect the findings and decide the workflow action. AWS states that these checks operate in detect mode, are limited to the policy scope, and do not replace prompt-injection controls [7].

9 Detailed Marking Workflow

9.1 Before assessment

- The module coordinator approves the assignment brief, rubric, weights, examples, exclusions, and feedback style.
- A protected calibration set contains representative submissions, difficult boundary cases, known misconceptions, and double-marked outcomes.
- The release manifest fixes the model, prompt, schema, parser, guardrail, retrieval corpus, and deterministic rule versions.
- CI/CD runs functional, security, privacy, repeated-run, and quality evaluations. Critical gates cannot be averaged away by high overall helpfulness.

9.2 During assessment

- The system creates a fresh, stateless assessment job with only the authorized artifact and approved context.
- The model produces criterion-level evidence before the score suggestion.
- The validation service rejects unsupported evidence, arithmetic errors, missing criteria, and prohibited content.
- The lecturer interface displays the original work, rubric, citations, suggestion, uncertainty, and failed checks together.
- The lecturer approves, edits, or rejects each criterion and records a reason when the official score differs materially.

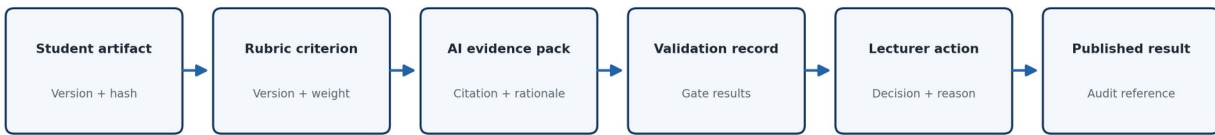
9.3 After assessment

- The published mark references the lecturer decision, not the raw model response.
- Student-facing feedback contains only approved content and excludes hidden system instructions or internal confidence labels.
- Appeals reopen an accountable workflow and preserve the original submission, manifest, suggestion, and decision.
- Production outcomes feed drift and error analysis without training on student work unless an explicitly governed data-use basis exists.

Decision	Required conditions
PASS_TO_REVIEW	Schema valid; all criteria resolved; citations valid; arithmetic valid; no critical safety or authorization failure.
RETRY	Transient model or parsing fault within a bounded retry budget; no business side effect occurred.
REJECT	Cross-student evidence, invalid rubric, unsupported claim, prohibited content, or policy contradiction.
MANUAL_ONLY	Low extraction fidelity, high model disagreement,

	missing source type support, or protected exception category.
PUBLISH	Authorized lecturer approval recorded and official result service accepts the deterministic payload.

Evidence chain from student work to defensible academic outcome



Every link is attributable, timestamped and correlation-ID traceable

Figure 7. The evidence chain supports lecturer review, student explanation, audit, and appeal by linking each published outcome to immutable inputs and attributable decisions.

10 Evaluation and Release Methodology

The evaluation should compare the proposed system against a human baseline using a protected, representative dataset. The unit of analysis is not only the final total. Criterion score, evidence correctness, rationale quality, subgroup performance, and operational behavior must be measured separately.

Dimension	Metric	Proposed release gate
Score alignment	Mean absolute error against adjudicated double marking	MAE $\leq$ 5 percentage points overall and within approved criterion bands.
Criterion agreement	Weighted kappa or intraclass correlation	Predefined minimum supported by confidence interval; investigate weak criteria.
Grounding	Citation precision and citation coverage	100% of score-bearing claims cited; $\geq$ 95% citation precision.
Hallucination	Unsupported material claim rate	0 critical fabricated claims; noncritical rate below approved threshold.
Consistency	Repeated-run score and reason variance	At least 95% of cases remain within one rubric band; unstable cases become manual.
Fairness	Error and override rate by approved subgroup and submission type	No unexplained material disparity; review with academic and governance owners.
Authority	Unauthorized publication attempts	Zero successful AI-initiated official mark publications.
Security	Cross-student retrieval or disclosure	Zero in negative tenant and role tests.
Performance	Submission acceptance latency	P95 $\leq$ 1 second over the stated peak-load window; AI processing measured separately.
Reliability	Accepted jobs reaching a terminal state	100% after reconciliation; no silent loss.
Cost	Cost per accepted and lecturer-approved suggestion	Measured by module, model, and rubric; budget alarm and routing policy applied.

10.1 Experimental design

17. Sample anonymized, ethically approved submissions across score bands, assignment types, and relevant student groups.
18. Obtain independent human marking and adjudication to establish a defensible reference.

19. Blind the AI workflow to student identity and irrelevant protected attributes.
20. Run each case repeatedly under fixed and alternative seeds or inference configurations to quantify variability.
21. Compare model versions and prompt or rubric variants without exposing the protected test set during tuning.
22. Analyze disagreement by criterion, artifact type, score band, subgroup, and model version.
23. Conduct lecturer usability testing: time saved, evidence usefulness, correction effort, and trust calibration.
24. Approve production only when all critical authority, privacy, security, and integrity gates pass.

Thresholds above are proposed acceptance targets, not reported experimental results. The institution should approve them according to assessment stakes, module context, cohort size, and existing human-marking reliability.

11 Failure Injection and Operational Evidence

Injected failure	Expected behavior	Evidence
Bedrock timeout or throttling	Submission remains accepted; bounded retry; visible PENDING state; manual fallback available.	Trace, retry counter, queue age, alarm, terminal reconciliation.
Duplicate event delivery	Idempotency returns the same job and prevents duplicate feedback or billing side effects.	Idempotency record and one official workflow instance.
Schema-invalid model output	No mark state changes; retry or manual review with reason code.	Validation log, captured manifest, rejected payload hash.
Hallucinated citation	Evidence gate rejects the criterion and blocks publication eligibility.	Citation-resolution failure and lecturer-visible alert.
Cross-student evidence attempt	Authorization denies retrieval and response release; security incident path triggers.	CloudTrail, denied request, correlation ID, alert.
Rubric version mismatch	Job stops before scoring and rebinds only through an authorized restart.	Manifest comparison and state transition log.
Queue backlog at deadline	Submission acceptance SLO remains protected; workers scale; oldest-message alarm fires.	API latency, queue age, concurrency, recovery time.
Lecturer portal unavailable	AI output remains unpublished; authorized user can resume review later.	No PUBLISHED event; durable review state.
Model regression after release	Canary gate or production alarm stops rollout and restores the last approved manifest.	Evaluation delta, rollback event, incident review.

The decisive production signal is not whether the model produced an answer. It is whether the system preserved the invariant when the answer was late, malformed, unsupported, unsafe, or unavailable.

12 Security Privacy and Fairness

12.1 Security and privacy

- Use Cognito groups and backend authorization; do not trust UI role hiding.
- Store artifacts with S3 Versioning, KMS encryption, restrictive bucket policies, and short-lived scoped access.
- Minimize student identifiers in prompts and logs; redact secrets and personal data before inference where feasible.

- Use per-job authorization context and never reuse model context or cached results across students without an identical security scope.
- Treat submission content as untrusted data. Isolate it from system instructions and tool authorization.
- Record model invocation and decision metadata without logging unnecessary student content.
- Define retention and deletion rules for raw artifacts, extracted text, model outputs, traces, and evaluation datasets.

12.2 Fairness and academic integrity

Fairness cannot be inferred from an overall accuracy score. The institution should analyze error, override, and uncertainty rates by legitimate educational categories, submission format, language characteristics, and accessibility accommodations where lawful and appropriate. The purpose is to detect whether the system systematically increases lecturer correction burden or grading error for particular groups.

Academic-integrity signals must remain advisory. A generative model should not make a misconduct finding because similarity, style, or unusual performance can have multiple explanations. A separate governed process should establish evidence, procedural fairness, and accountable human judgment.

13 Implementation Roadmap

Phase	Scope	Exit evidence
1 Frame and contract	Actors, outcomes, invariants, exclusions, OpenAPI, event schema, state machine, threat model.	Approved design pack and acceptance criteria.
2 Deterministic portal foundation	Authenticated submission, S3 version, idempotency, status, audit, manual marking.	Peak-load, authorization, durability, and recovery tests.
3 Offline AI evaluation	Build evidence-pack schema, prompts, protected datasets, repeated-run and security evaluations.	Evaluation report; no student-facing release.
4 Shadow workflow	Generate suggestions beside normal marking without influencing official outcomes.	Human-AI agreement, correction effort, fairness, latency, and cost evidence.
5 Limited assisted release	Lecturer-facing suggestions for selected modules with mandatory approval and monitoring.	Incident drills, rollback proof, adoption and quality review.
6 Controlled scale	Expand only approved assignment types; re-evaluate every material model, rubric, or workflow change.	Continuous evidence and governance sign-off.

13.1 Responsibilities

Role	Accountability
Academic owner	Rubric validity, marking policy, approval workflow, appeals, educational impact.
Product owner	Business outcomes, scope, user experience, prioritization, and operational readiness.
AI or evaluation lead	Datasets, model and prompt evaluation, drift, limitations, and release evidence.
Security and privacy	Threat model, data handling, authorization, logging, retention, and incident response.
Platform team	Contracts, orchestration, reliability, observability, cost, and rollback.
Lecturer	Review, official decision, override reason, exception handling, and feedback quality.
Governance body	Risk acceptance, critical thresholds, fairness review, and continued-use decision.

13.2 IdeaFarm GitHub project enhancement roadmap

The repository should evolve through small, testable pull requests rather than a single architectural rewrite. Each phase below preserves the working Streamlit, S3, SQS, Lambda, Bedrock, DynamoDB, CodeQL, and WAF foundation while adding one enforceable control boundary. Priority P0 blocks any production assessment use; P1 is required before reliable peak-load processing; P2 and P3 are required before AI suggestions can participate in an official marking workflow; P4 supports controlled scale.

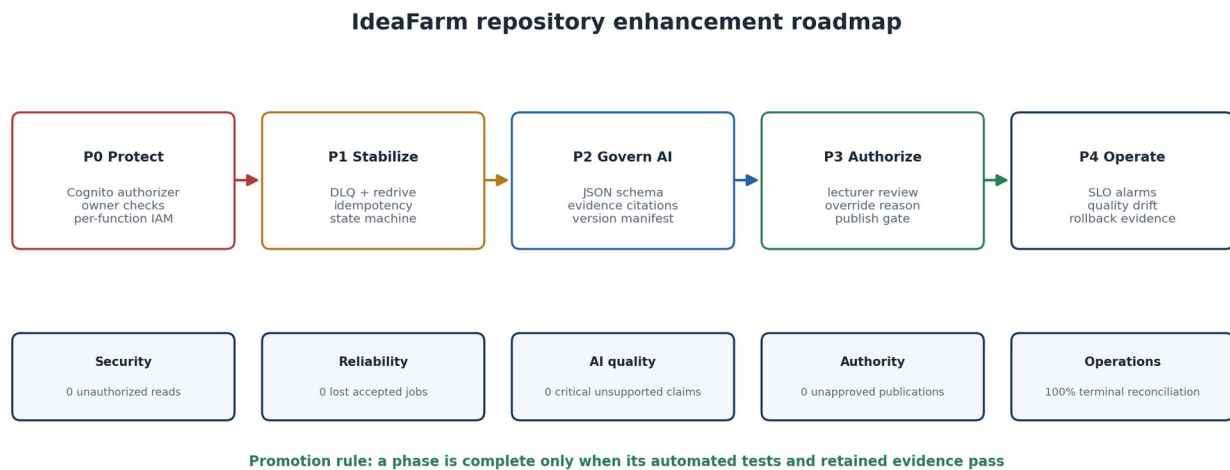


Figure 8. Repository enhancement sequence. Security and ownership come first, followed by durable processing, governed AI output, lecturer authority, and continuous operational evidence.

Priority	Repository components	Required enhancement	Acceptance evidence
P0 Identity	cloudformation/infr.yaml; API methods; Streamlit login	Create a Cognito user-pool authorizer; require JWTs on upload, poll and dashboard endpoints; derive student identity only from verified claims; authorize every submission and report read by owner or lecturer role.	Negative tests show zero anonymous, cross-student or student-to-dashboard access; CloudTrail and application logs contain actor, role, submission ID and correlation ID.
P0 Least privilege	GraderLambdaRole and all Lambda definitions	Replace the shared role with separate upload, grading, result and dashboard roles. Scope S3 prefixes, DynamoDB actions, queue actions and Bedrock Agent alias permissions. Remove wildcard permissions where service support permits.	IAM Access Analyzer findings reviewed; policy simulation passes permitted actions and denies unrelated bucket, table, queue and model access.

P0 Storage protection	SourceBucket; ReportBucket; DynamoDB	Enable S3 Block Public Access, versioning, KMS encryption, ownership enforcement, lifecycle and access logging. Add DynamoDB point-in-time recovery, KMS encryption and deletion protection for production.	Infrastructure tests verify every property; unauthorized public access fails; restore exercise recovers a known object version and table record.
P0 Upload integrity	get_signed_url Lambda; upload page	Bind upload URLs to authenticated identity, normalized assignment ID, file size, content type and checksum. Create the submission record before upload and verify the completed object before emitting an accepted event. Quarantine archives until malware and ZIP safety checks pass.	Wrong checksum, oversized archive, path traversal, ZIP bomb, wrong media type and replay tests all fail safely; valid upload returns one durable receipt.
P1 Queue reliability	S3 and EventBridge route; SQS; event source mapping	Define EventBridge rule and queue policy in infrastructure as code. Add a DLQ and redrive policy, visibility timeout greater than maximum processing time, bounded batch size, partial-batch failure handling and reserved concurrency aligned with Bedrock quotas.	Failure injection proves bounded retries, DLQ arrival and replay; 100% of accepted jobs reach a terminal state after reconciliation.
P1 Idempotent workflow	grading Lambda; DynamoDB item model	Use eventId and submissionId as idempotency keys. Replace unconditional put with conditional state transitions: ACCEPTED, QUEUED, PROCESSING, VALIDATING, REVIEW, APPROVED or REJECTED, and PUBLISHED. Hash the source version and result payload.	Duplicate SQS delivery produces one logical grading job, one report version and no duplicate model billing or publication side effect.
P2 AI contract	agent prompt; grading rules; grader Lambda	Replace permissive raw-text fallback with a versioned JSON Schema. Require rubric criterion ID, suggested score, exact file and line evidence, rationale, uncertainty and model-prompt-rubric	Contract suite rejects missing, extra and wrong-type fields; 100% of score-bearing claims resolve to the submitted artifact; critical unsupported claims equal zero.

		manifest. Route malformed output to retry or manual review.	
P2 Deterministic validation	new validation module or Lambda	Recalculate weights and totals in code; verify evidence hashes and rubric version; scan output for sensitive data and prompt attacks; apply score bounds and manual-review thresholds. Bedrock output remains an untrusted candidate.	Golden-set, mutation and adversarial tests produce stable reason codes; arithmetic is exact; validation failures never change the official result state.
P3 Lecturer authority	instructor pages; result service; DynamoDB audit records	Add criterion-level review with original code, CodeQL findings, citations, suggestion and uncertainty together. Require approve, edit or reject actions and an override reason. Create a separate publication endpoint unavailable to the AI execution role.	Every published result has an authenticated lecturer decision, rubric version, candidate hash and timestamp; zero direct AI-initiated publications.
P4 Operations and release	CloudWatch; CI/CD; CodeQL workflows; deployment configuration	Emit structured logs and embedded metrics for queue age, processing latency, Bedrock throttling, schema failures, citation failures, override rate, cost and drift. Add alarms, dashboards, canary deployment, evaluation gates and rollback to a published Lambda and prompt-model manifest.	SLO dashboard and alarms are exercised; canary failure automatically stops promotion; rollback restores the last approved manifest within the recovery objective.

13.3 Recommended pull request sequence

25. PR 1 adds Cognito authorization, ownership checks, correlation IDs and negative authorization tests.
26. PR 2 separates IAM roles and hardens S3 and DynamoDB without changing the user workflow.
27. PR 3 codifies the EventBridge-to-SQS route, DLQ, redrive, visibility timeout, partial-batch response and idempotent job state.
28. PR 4 introduces the versioned AI suggestion schema, deterministic validator and fail-closed handling.
29. PR 5 adds lecturer review, override reasons, a publication service and immutable audit events.

30. PR 6 adds automated load and failure tests, model-quality evaluation, dashboards, alarms, canary deployment and rollback evidence.

Each pull request should include an architecture decision record, updated threat model, infrastructure diff, automated tests, failure-injection result, operational metric or alarm, rollback instruction, and a short evidence manifest. This makes the GitHub history itself part of the assurance record.

13.4 Shift-left measurable contracts and retained evidence

Shift-left design moves verification decisions to the earliest point at which they can be made economically and objectively. In this project, a requirement is not ready for implementation until it states the protected business outcome, invariant, measurement window, threshold, failure behavior, test oracle, evidence location, accountable owner, and release consequence. NIST SSDF recommends integrating secure practices throughout the software life cycle, while the AWS Generative AI Lens and AWS CAF for AI emphasize evaluation, governance, observability, and lifecycle controls rather than treating model deployment as the end of assurance [6,9,12].

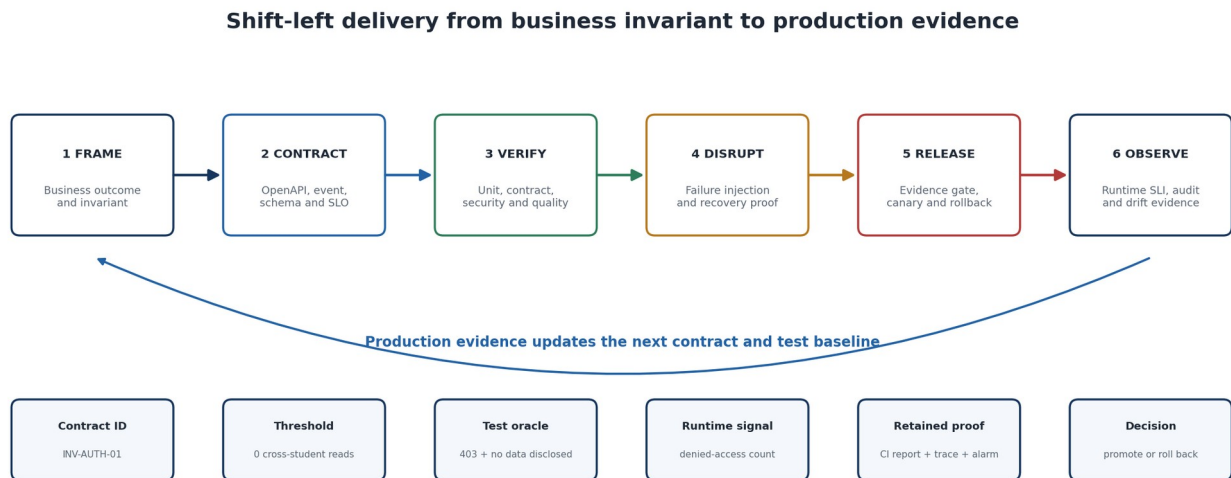


Figure 9. The measurable contract is designed before implementation, exercised under deliberate failure, enforced at release, and continuously checked with production evidence.

The contract is executable at several layers. OpenAPI and JSON Schema constrain interfaces; state-transition and authorization rules constrain business behavior; tests establish the expected pass or fail result; deployment policy blocks promotion when evidence is absent; and runtime telemetry demonstrates whether the invariant continues to hold. OWASP ASVS provides a basis for verifiable application-security requirements, and W3C Trace Context supports correlation across distributed components so that API, queue, model, validation, review, and publication evidence can be joined without relying on narrative reconstruction [13,14].

Invariant and contract	Early verification	Failure injection	Production evidence	Release rule
AUTH-01 Student	API authorization	Replay a valid token	Structured denial	Any disclosure

isolation: a student can read only submissions bound to the verified subject claim. Target: zero cross-student disclosures.	tests use two students, a lecturer, expired tokens, altered object identifiers and direct endpoint calls. Oracle: unauthorized calls return 403 and disclose no metadata.	against another student submission; bypass the UI; remove an ownership condition in a test deployment.	logs, actor and resource identifiers, CloudTrail records, trace ID and a zero-disclosure security test report.	blocks promotion. A production violation revokes access, opens an incident and rolls back the release.
DUR-01 Accepted work is never lost. Target: 100% of accepted receipts reconcile to one terminal state within the agreed recovery window.	Contract and integration tests verify durable receipt creation before event emission, idempotent transitions and reconciliation.	Throttle Bedrock, terminate a Lambda invocation, duplicate and delay SQS messages, exhaust retries and replay the DLQ.	Receipt-to-terminal reconciliation metric, queue age, DLQ depth, retry count, state history and recovery-run record.	Unreconciled accepted work or an untested DLQ blocks release; breach invokes reconciliation and manual marking.
AI-01 Every score-bearing suggestion has valid, resolvable evidence. Target: zero critical unsupported claims in the protected evaluation set.	Schema, citation resolver, evidence-hash, rubric-version, golden-set, mutation and adversarial tests run before merge.	Return malformed JSON, invented files or lines, conflicting rubric versions, prompt-injected content and out-of-range scores.	Schema-failure rate, citation-resolution rate, unsupported-claim rate, reason codes, model-prompt-rubric manifest and evaluation report.	Malformed or unsupported output cannot enter lecturer review as a valid candidate; threshold failure blocks the model manifest.
AUTHORITY-01 AI cannot publish an official result. Target: zero publications without an authenticated lecturer decision.	IAM simulation, API negative tests and state-machine tests prove that only the publication service can transition APPROVED to PUBLISHED.	Invoke publication with the AI role, omit the decision record, alter the candidate hash and replay an approval.	Lecturer identity, decision, override reason, candidate hash, rubric version, timestamp and immutable publication audit event.	Any AI-initiated or unaudited publication is a critical incident and disables publication until integrity is restored.
SLO-01 Assisted marking remains responsive and bounded. Target: agreed P95 job latency and error budget measured over a defined window.	Load tests model submission bursts, service quotas, payload sizes and concurrency before deployment.	Inject downstream latency, Bedrock throttling, queue backlog and partial regional dependency failure.	P50/P95/P99 latency, queue age, throttles, error rate, cost per completed job, alarm history and canary comparison.	Canary promotion stops when the SLO or error-budget gate fails; traffic returns to the last approved manifest.

The evidence manifest should be machine readable and stored with the release. At minimum it records the commit and infrastructure version, contract identifiers, test suite and dataset versions, model and prompt manifest, security scan results, failure experiments, SLO comparison, approvers, deployment identifier, and rollback target. Evidence has a freshness period: a model, prompt, rubric, parser, policy, infrastructure, or material dependency change

invalidates the affected evidence and triggers re-evaluation. This converts continuous delivery into continuous assurance rather than a one-time approval exercise.

For teaching, students should be assessed on the traceability chain rather than the quantity of diagrams or tests. A defensible submission demonstrates: business problem to invariant; invariant to executable contract; contract to positive and negative tests; tests to failure experiments; experiments to observable signals; signals to retained evidence; and evidence to an explicit release or rollback decision. The contract is therefore both a design artifact and a marking rubric for engineering judgment.

14 Discussion Limitations and Conclusion

14.1 Discussion

The architecture reframes AI marking as evidence preparation rather than autonomous judgment. This is operationally important. A portal can tolerate variable wording if the official state transition is stable. It can tolerate occasional low confidence if the failure is visible and manual work remains possible. It cannot tolerate silent data loss, unauthorized access, fabricated evidence, incorrect arithmetic, or unapproved publication.

The approach also clarifies where formal verification may help. Automated Reasoning checks can validate explicit rules derived from a clear policy, such as whether required rubric conditions are logically satisfied. They cannot determine all pedagogical quality, protect against every prompt injection, or validate facts outside their policy variables. They should therefore complement, not replace, schema checks, conventional code, security controls, evaluation, and lecturer judgment [7,8].

14.2 Limitations

- The proposed thresholds require empirical calibration with the institution's assignments and human-marking baseline.
- Open-ended creative work may resist reliable criterion-level automation more than structured programming tasks.
- Research evidence evolves rapidly and results may not transfer across disciplines, languages, cohorts, or model versions.
- Human approval reduces but does not eliminate bias or inconsistency; lecturer decisions also require moderation and quality assurance.
- Automated Reasoning support, language, Regions, latency, and service features may change and must be verified before implementation.

14.3 Conclusion

A trustworthy Student Portal should not attempt to make a probabilistic model deterministic through prompting. It should make the business application deterministic around the model. Immutable source artifacts, versioned rubrics, typed evidence packs, deterministic arithmetic,

explicit states, safe failure, independent evaluation, and lecturer approval convert uncertain model output into a controlled recommendation workflow.

The final design rule is simple: AI may accelerate interpretation, but academic authority remains with governed code and accountable people. Delivery is complete only when measurable evidence shows that this rule remains true during ordinary operation and deliberate failure.

References

- [1] Tseng, E.-Q. et al. (2025). CodEv: An Automated Grading Framework Leveraging Large Language Models for Consistent and Constructive Feedback. arXiv:2501.10421. [Source](#)
- [2] Jukiewicz, M. (2025). A systematic comparison of Large Language Models for automated assignment assessment in programming education. arXiv:2509.26483. [Source](#)
- [3] Zhou, Q. et al. (2026). Impacts of Histories and Models on LLM Grading: A Study in Advanced Software Engineering Courses. arXiv:2606.08400. [Source](#)
- [4] Sun, W. et al. (2026). EDU-CIRCUIT-HW: Evaluating Multimodal Large Language Models on Real-World University-Level STEM Student Handwritten Solutions. arXiv:2602.00095. [Source](#)
- [5] NIST (2024). Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. NIST AI 600-1. [Source](#)
- [6] Amazon Web Services (2025). AWS Well-Architected Framework: Generative AI Lens. [Source](#)
- [7] Amazon Web Services. What are Automated Reasoning checks in Amazon Bedrock Guardrails? [Source](#)
- [8] Amazon Web Services. Detect and filter harmful content by using Amazon Bedrock Guardrails. [Source](#)
- [9] Amazon Web Services (2024). AWS Cloud Adoption Framework for Artificial Intelligence, Machine Learning, and Generative AI. [Source](#)
- [10] Amazon Web Services. Create an Automated Reasoning policy. [Source](#)
- [11] Ji Pengxiang. IdeaFarm Secure Code Lecturer public repository. Reviewed at commit 1e387b54a0eba7cb18bc1a887f1d35c78849f7d0. [Source](#)
- [12] National Institute of Standards and Technology (2022). Secure Software Development Framework Version 1.1. NIST SP 800-218. [Source](#)
- [13] OWASP Foundation. Application Security Verification Standard. [Source](#)
- [14] World Wide Web Consortium (2021). Trace Context Level 1 W3C Recommendation. [Source](#)
- [15] Amazon Web Services. Operational Excellence Pillar Design Principles. [Source](#)

Appendix A Contract Examples

A.1 Plain English contract

- When an authorized student submits a valid artifact, the portal records one immutable version and returns a durable receipt.
- The AI service may prepare evidence-linked marking suggestions only from the authorized submission, assignment brief, and active rubric.
- The portal rejects suggestions that lack citations, violate schema, use the wrong rubric, fail arithmetic, cross student boundaries, or contain prohibited content.
- A lecturer must approve or override the suggestion before the official result can be published.
- If AI processing fails, the accepted submission remains safe and the workflow exposes PENDING, REVIEW, or FAILED rather than inventing completion.

A.2 Event contract extract

Field	Example	Rule
eventType	assessment.job.accepted	Versioned event name.
eventId	UUID	Globally unique for deduplication.
correlationId	SUB-2026-000184	Present in API, queue, model, validation, and audit records.
submissionId	UUID	Resolves to one student-owned immutable artifact.
artifactVersionId	S3 version ID	Never replaced during the job.
rubricVersion	RUBRIC-3.2	Approved and immutable for the job.
manifestId	MANIFEST-2026-09-17-A	Binds model, prompt, schema, guardrail, parser, and rules.
occurredAt	ISO 8601 timestamp	Generated by the accepting service.

A.3 Audit record minimum

- Actor identity and role.
- Submission, artifact version, rubric, and manifest identifiers.
- AI candidate hash and deterministic validation results.
- Lecturer-approved criterion scores and feedback version.
- Override or rejection reason.
- State transition, timestamp, correlation ID, and originating request.
- Release or rollback event and operational evidence link.

Appendix B Acceptance Criteria

Contract area	Acceptance criterion
Durability	100% of accepted submissions resolve to a terminal state; zero lost artifacts in failure testing.
Idempotency	Repeated identical requests create one logical submission and one assessment job.
Authorization	Zero cross-student or unauthorized lecturer disclosures in negative tests.
Grounding	Every score-bearing claim contains a valid citation; critical fabricated claims equal zero.
Arithmetic	Deterministic recomputation exactly matches the official criterion total.
Authority	Zero official publications without an attributable lecturer approval event.
Latency	P95 submission acceptance ≤ 1 second during the stated peak load; AI latency reported separately.
Recovery	Backlog and injected dependency failures recover within the approved operational objective.
Fairness	No unexplained material error or override disparity across approved analysis groups.
Change control	Every material model, prompt, rubric, schema, rule, or guardrail change creates a new evaluated manifest.